

FUTUREX

Technical Reference

**Excrypt Direct Key Injection Protocol
API Specifications**

THIS DOCUMENT CONTAINS CONFIDENTIAL INFORMATION PROPRIETARY TO FUTUREX, LP. ANY UNAUTHORIZED USE, DISCLOSURE, OR DUPLICATION OF THIS DOCUMENT OR ANY OF ITS CONTENTS IS EXPRESSLY PROHIBITED.

Excrypt Key Injection Protocol

Introduction

The purpose of this document is to outline a serial protocol for injection of cryptographic keys into the terminals that do not define their own protocol. It is intended to be freely available for implementation by vendors who wish to have their terminals interface with the SKI Series, which is used for injecting defined symmetric keys into devices.

The SKI Series supports the following serial baud rates and data formats:

Baud rates: 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200

Data formats: 7 data bits, even parity, 1 stop bit (7E1), or 8 data bits, no parity, 1 stop bit (8N1)

As a reference, the most common serial port setting for currently supported terminals would be:

Baud Rate: 1200

Data Bits: 7

Parity: Even

Stop Bits: 1

General Message Format

The core form of a message packet is outlined below. It is used for messages sent to and received from terminals.

`<STX> [MESSAGE] <ETX> { LRC }`

Where:

`<STX>` is the “start transmission” byte `0x02`.

`[MESSAGE]` is ASCII text containing data.

`<ETX>` is the “end transmission” byte `0x03`.

`{ LRC }` is a single-byte longitudinal redundancy check built from XOR-ing all the bytes from the first byte in `[MESSAGE]` up to and including the `<ETX>` byte.

Communication Flow

When the SKI Series or the terminal receives a packet, it will calculate the LRC and compare it to the one sent with the packet. If the LRCs match, a single acknowledge <ACK> byte (0x06) is sent in response. If the LRCs do not match, a negative-acknowledge <NAK> byte (0x15) is sent.

After sending a message:

If a <NAK> is received, the previous message should be sent again. After three attempts and three <NAK> responses, communication should be stopped.

If an <ACK> is received, the message was successfully transmitted.

If a response message is also expected, the sender should wait for the response, acknowledge (ACK) or negative-acknowledge (NAK) it, and then continue to send its own next message.

Typical Communication Flow - Example:

SKI Series	Direction	terminal
Message Packet	----->	
	<-----	<ACK> = LRC OK
	<-----	Response Packet
<ACK> = LRC OK	----->	

Next Message Packet...

Terminal Receives Bad Packets from SKI Series - Example:

SKI Series	Direction	terminal
Message Packet	----->	
	<-----	<NAK> = BAD LRC
Same Message Packet	----->	
	<-----	<NAK> = BAD LRC
Same Message Packet	----->	
	<-----	<NAK> = BAD LRC
Stop Communication		
"Error Too many NAKs"		

SKI Series Receives Bad Packets From Terminal - Example:

SKI Series	Direction	terminal
Message Packet	----->	
	<-----	<ACK>
	<-----	Response Packet
<NAK> = BAD LRC	----->	
	<-----	Same Response Packet
<NAK> = BAD LRC	----->	
	<-----	Same Response Packet
<NAK> = BAD LRC	----->	
		Stop Communication "Error"

Serial Number Read Command

The SKI Series will send the following commands to request a 16-character serial number from the terminal. The command is within the previously described packet format, and the terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII decimal values are characters '0'-'9', while valid ASCII hexadecimal values are characters '0'-'9' and 'A'-'F'.

Retrieve Serial Number (03)

Message Format

Field	Size or Location	Title	Description
1	Bytes 0-1	Command	“03” Read Serial Number Command
2	Bytes 2-3	Command Version	Version of command for expandability of command in ASCII hex “01”

Response Format

Field	Size	Title	Description
1	Bytes 0-1	Command	“03” Read Serial Number Command Response
2	Bytes 2-3	Key Success Acknowledgement	2 digit key injection response code in ASCII decimal; “00” indicates success *
3	Bytes 4-19	Serial Number	16-character serial number

*Only the “00” response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

Serial Number Read Example

```
Terminal Serial Number: 123456789ABCDEFGG

SKI Series ---> terminal
<0x02>0301<0x03><0x0E>

SKI Series <--- terminal
<0x06>
<0x02>0300123456789ABCDEFGG<0x03><0x01>

SKI Series ---> terminal
<0x06>
```

Serial Number Write Command

The SKI Series will send the following commands to write a 16 character serial number to the terminal. The command is within the previously described packet format. The terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII dec values are characters '0'-'9', while valid ASCII hex values are characters '0'-'9' and 'A'-'F'.

Write Serial Number (04)

Message Format

Field	Size or Location	Title	Description
1	Bytes 0-1	Command	“04” write serial number command
2	Bytes 2-3	Command Version	Version of command for expandability of command in ASCII hex “01”
3	Bytes 4-19	Serial Number	16-character serial number

Response Format

Field	Size	Title	Description
1	Bytes 0-1	Command	“03” retrieve serial number command response
2	Bytes 2-3	Key Success Acknowledgement	2 digit key injection response code in ASCII dec; “00” indicates success *

*Only the “00” response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

Serial Number Write Example

```
Initial Serial Number: 123456789ABCDEFG
```

```
SKI Series ---> terminal
<0x02>0401123456789ABCDEFG<0x03><0x0E>
```

```
SKI Series <--- terminal
<0x06>
<0x02>0400<0x03><0x01>
```

```
SKI Series ---> terminal
<0x06>
```

Legacy Symmetric Key Injection Commands

The SKI Series support the legacy symmetric key injection commands 01 and 02 for clear key DUKPT and master session key injection.

To inject symmetric keys into the terminal, the SKI Series will send the following commands within the previously described packet format. The terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII dec values are characters '0'-'9', while valid ASCII hex values are characters '0'-'9' and 'A'-'F'.

Inject DUKPT Key (00)

Message Format

Bytes 0-1	“00” inject DUKPT key command
Bytes 2-3	2 digit key slot number in ASCII hex, “00” if device has only one DUKPT key location
Bytes 4-23	20 digit key serial number (KSN) in ASCII hex
Bytes 22-37, 22-53, or 22-69	16, 32, or 48 digit DUKPT Initial Key in ASCII hex

Response Format

Bytes 0-1	“00” inject DUKPT key command response
Bytes 2-3	2 digit KSN injection response in ASCII dec; “00” indicates success
Bytes 4-5	2 digit DUKPT key injection response in ASCII dec; “00” indicates success*
(and preferably) Bytes 6-9	4 digit ASCII hex checksum**

* Only the “00” response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

** The checksum is calculated by encrypting an all-zero array (which is the same length as the key) with the recently sent key. The first four hex characters of the result is the checksum

Inject Master Session Key (01)

Message Format

Bytes 0-1	"01" inject master session key command
Bytes 2-3	2 digit key slot number in ASCII hex, "00" if device has only one master session key location
Bytes 4-19, 4-35, or 4-51	16, 32, or 48 digit master session key in ASCII hex

Response Format

Bytes 0-1	"01" inject master session key command response
Bytes 2-3	2 digit master session key injection response in ASCII dec; "00" indicates success
(and preferably) Bytes 4-7	4 digit ASCII hex checksum*

* Only the "00" response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

** The checksum is calculated by encrypting an all zero string, the same length as the key, with the recently sent key. The first four hex characters of the result is the checksum.

Example Legacy Key Injections

Double Length DUKPT Key and KSN:

```

DUKPT KSN: FFFFA987654321000000
DUKPT Key: 11111111111111112222222222222222
Checksum : D2B9

SKI Series ---> terminal
<0x02>0000FFFA9876543210000001111111111111222222222222222<0
x03><0x73>

SKI Series <--- terminal
<0x06>
<0x02>000000D2B9<0x03><0x0E>

SKI Series ---> terminal
<0x06>

```

Double Length Master Session Key:

```

Master Session Key: 11111111111111112222222222222222
Checksum : D2B9

SKI Series ---> terminal
<0x02>01001111111111111111112222222222222222<0x03><0x02>

SKI Series <--- terminal
<0x06>
<0x02>0100D2B9<0x03><0x0F>

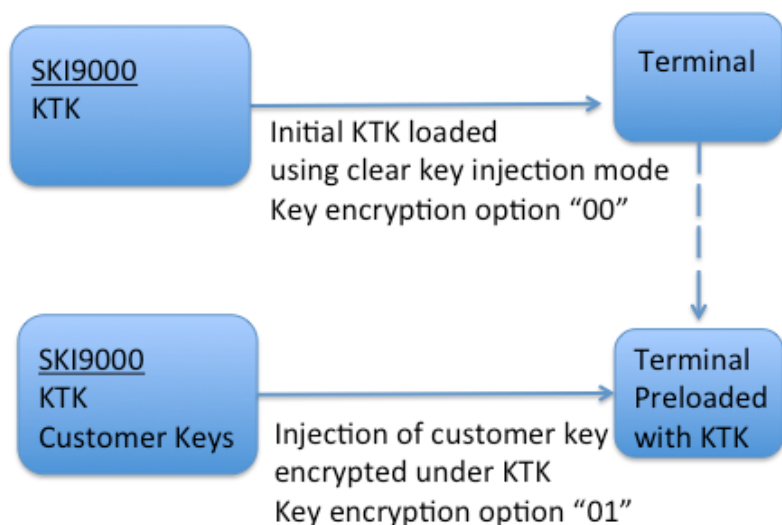
SKI Series ---> terminal
<0x06>

```

Symmetric Key Injection Commands

The SKI Series 02 command provides a method for securing the key injection messages between the SKI Series and the terminal being loaded. The typical scenario involves loading a key transfer key (KTK) onto the terminal on the manufacturing floor. This key is often only able to be loaded once or by some other means not accessible after leaving manufacturing. command 02 with the clear key encryption option “00” will accomplish this task.

After the terminal leaves the manufacturing environment, the terminal is preloaded with a KTK. Now command 02 with the preloaded KTK encryption option “01” will inject the terminal with all desired keys encrypted under the preloaded KTK.



To inject symmetric keys into the terminal, the SKI Series will send the following commands within the previously described packet format. The terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII dec values are characters '0'-'9', while valid ASCII hex values are characters '0'-'9' and 'A'-'F'.

Inject DUKPT Key (00)

Message Format

Bytes 0-1	“00” inject DUKPT key command
Bytes 2-3	2 digit key slot number in ASCII hex, “00” if device has only one DUKPT key location
Bytes 4-23	20 digit key serial number (KSN) in ASCII hex
Bytes 22-37, 22-53, or 22-69	16, 32, or 48 digit DUKPT initial key in ASCII hex

Response Format

Bytes 0-1	“00” inject DUKPT key command response
Bytes 2-3	2 digit KSN injection response in ASCII dec; “00” indicates success
Bytes 4-5	2 digit DUKPT key injection response in ASCII dec; “00” indicates success
(and preferably) Bytes 6-9	4 digit ASCII hex checksum*

* Only the “00” response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

** The checksum is calculated by encrypting an all-zero array (which is the same length as the key) with the recently sent key. The first four hex characters of the result is the checksum.

Inject Key Under KTK (02)

To inject symmetric keys into the terminal, the SKI Series will send the following commands within the previously described packet format. The terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII decimal values are characters '0'-'9', while valid ASCII hex values are characters '0'-'9' and 'A'-'F'.

Message Format

Field	Size or Location	Title	Description
1	Bytes 0-1	Command	“02” inject key under KTK command
2	Bytes 2-3	Command Version	Version of command for expandability of command in ASCII hex “01”
3	Bytes 4-5	Key Slot Storage	2 digit key slot number in ASCII hex, “00” if device has only one key location
4	Bytes 6-7	KTK Slot Storage	2 digit KTK slot number in ASCII hex, “00” if device has only one key location or if no KTK is sent.
5	Bytes 8-9	Key Type	Identifier of key type in ASCII hex. See Table 1.
6	Bytes 10-11	Key Encryption	Identifier if key is clear key load (“00”), encrypted under a preloaded KTK (“01”), or clear KTK (“02”)
7	Bytes 12-15	Key Checksum	4 digit ASCII hex checksum of incoming key*
8	Bytes 16-19	KTK Checksum	4 digit ASCII hex checksum KTK* (All zeros if KTK is not used in key transfer)
9	Bytes 20-39	KSN Serial Number	20 digit key serial number (KSN) in ASCII hex (All zeros if KSN is not used in key type)
10	Bytes 40-42	Length of Key	Length of incoming key or key block in ASCII hex (“010”, “020”, “030”, etc.)
11	Bytes vary according to Field 10	Key Or Key Block	16, 32, or 48 digit ASCII hex key or TR-31A key block (Length determined by field 10)
12	3 Bytes	Length of KTK	Length of KTK in ASCII hex (“010”, “020”, “030”) (All zeros if field 6 is not 02)
13	Bytes vary according to Field 12	KTK	16, 32, or 48 digit ASCII hex key (Only used if field 6 is 02) (Length determined by field 12)

Response Format

Field	Size	Title	Description
1	Bytes 0-1	Command	"02" inject key under KTK command response
2	Bytes 2-3	Key Success Acknowledgement	2 digit key injection response code in ASCII dec; "00" indicates success
3	Bytes 4-7	Checksum	4 digit ASCII hex checksum**

* Only the "00" response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

** The checksum is calculated by encrypting an all zero string, the same length as the key, with the recently sent key. The first four hex characters of the result create the checksum.

Example Key Injection

Double Length DUKPT 3DES BDK Key and KSN Injection with Preloaded KTK:

Key Slot = 1, KTK Slot = 0, Key Type = DUKPT 3DES BDK

```

DUKPT 3DES BDK : 0123 4567 89AB CDEF FEDC BA98 7654 3210
BDK Checksum   : 08D7
DUKPT KSN      : FFFF 9876 5432 10E0 0000
Derived IPEK   : 6AC2 92FA A131 5B4D 858A B3A3 D7D5 933A
IPEK Checksum  : AF8C
KTK KEY        : 0123 4567 8901 2345 6789 0123 4567 8901
KTK Checksum   : 0596
IPEK encrypted under KTK: 6C94 3EEE ACB6 6961 9FCB AB2C 2270 7694

SKI Series ---> terminal
<0x02> 0201 0100 0801 AF8C 0596 FFFF 9876 5432 10E0 0000 0206 C943
EEEE CB66 9619 FCBA B2C2 2707 6940 00<0x03><0x7F>

SKI Series <--- terminal
<0x06>
<0x02> 0200 AF8C <0x03><0x01>

SKI Series ---> terminal
<0x06>

```

Example Key Injections

Double Length DUKPT Key and KSN:

```

DUKPT KSN: FFFFA987654321000000
DUKPT Key: 11111111111111112222222222222222
Checksum : D2B9

SKI Series ---> terminal
<0x02>0000FFFA9876543210000001111111111111222222222222222<0
x03><0x73>

SKI Series <--- terminal
<0x06>
<0x02>000000D2B9<0x03><0x0E>

SKI Series ---> terminal
<0x06>

```

Double Length Master Session Key:

```

Master Session Key: 11111111111111112222222222222222
Checksum : D2B9

SKI Series ---> terminal
<0x02>01001111111111111111112222222222222222<0x03><0x02>

SKI Series <--- terminal
<0x06>
<0x02>0100D2B9<0x03><0x0F>

SKI Series ---> terminal
<0x06>

```

Inject Key Under KTK Commands

To inject symmetric keys into the terminal, the SKI Series will send the following commands within the previously described packet format. The terminal will send back the corresponding response packets.

NOTE: Values contained in “quotes” are considered ASCII character arrays, not strings terminated by the '\0' character. Valid ASCII dec values are characters '0'-'9', while valid ASCII hex values are characters '0'-'9' and 'A'-'F'.

Inject Key Under KTK (02)

Message Format

Field	Size or Location	Title	Description
1	Bytes 0-1	Command	“02” inject key under KTK command
2	Bytes 2-3	Command Version	Version of command for expandability of command in ASCII hhex “01”
3	Bytes 4-5	Key Slot Storage	2 digit key slot number in ASCII hex, “00” if device has only one key location
4	Bytes 6-7	KTK Slot Storage	2 digit KTK slot number in ASCII hex, “00” if device has only one key location or if no KTK is sent.
5	Bytes 8-9	Key Type	Identifier of key type in ASCII hex. See Table 1.
6	Bytes 10-11	Key Encryption	Identifier if key is clear key load (“00”), encrypted under a preloaded KTK (“01”), or clear KTK (“02”)
7	Bytes 12-15	Key Checksum	4 digit ASCII hex checksum of incoming key*
8	Bytes 16-19	KTK Checksum	4 digit ASCII hex checksum KTK* (All zeros if KTK is not used in key transfer)
9	Bytes 20-39	KSN Serial Number	20 digit Key Serial Number (KSN) in ASCII hex (All zeros if KSN is not used in key type)
10	Bytes 40-42	Length of Key	Length of incoming key or key block in ASCII hex (“010”, “020”, “030”, etc.)
11	Bytes vary according to Field 10	Key Or Key Block	16, 32, or 48 digit ASCII hex key or TR-31A key block (Length Determined by Field 10)
12	3 Bytes	Length of KTK	Length of KTK in ASCII hex (“010”, “020”, “030”) (All zeros if Field 6 is not 02)
13	Bytes vary according to Field 12	KTK	16, 32, or 48 digit ASCII hex key (Only used if Field 6 is 02) (Length Determined by Field 12)

Response Format

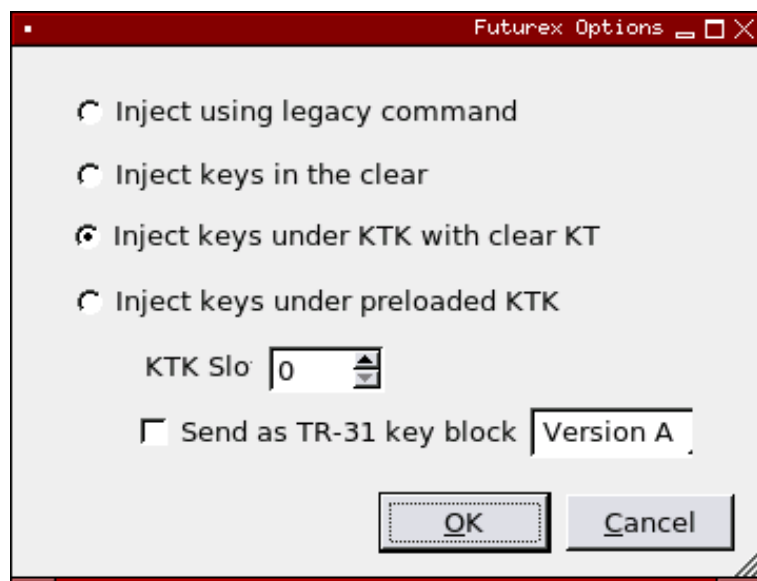
Field	Size	Title	Description
1	Bytes 0-1	Command	"02" inject key under KTK command response
2	Bytes 2-3	Key Success Acknowledgement	2 digit key injection response code in ASCII dec; "00" indicates success, "01" is key load failure, "02" indicates KTK failure, "03" indicates KSN failure
3	Bytes 4-7	Checksum	4 digit ASCII hex Checksum*

* Only the "00" response code is reserved, which indicates success. Any other code may be used for error diagnosis. The SKI Series will pass the error code to the user display window.

** The checksum is calculated by encrypting an all zero string, the same length as the key, with the recently sent key. The first four hex characters of the result create the checksum.

Futurex Protocol Device Options

The SKI Series has the ability to define protocol specific options.



- Inject using legacy command – Sends the key using the inject DUKPT key command 00, or inject master session key command 01.
- Inject keys in the clear – Sends the key using the keys under KTK 02 command but does so in clear text.
- Inject keys under KTK with clear KTK – Sends the key under the KTK along with the clear KTK. Both are sent in the 02 command message. The KTK must be defined under a device on the SKI POS Terminal window at the location specified by the KTK slot selection box.
- Inject keys under preloaded KTK – The key will be sent under KTK, but the KTK will not be sent. The KTK must be defined under a device on the SKI POS terminal window at the location specified by the KTK slot selection box.
 - Send as TR-31 - If selected, the key under the KTK will be sent as a TR-31 key block. The dropdown box selection determines if the TR-31 key block will be Version A (Key Variant Method 2005) or Version B (Key Derivation Method).
 - KTK Slot – This selection box identifies the KTK used to encrypt the key being sent. The KTK Slot must match the location of the KTK in the SKI POS terminal window. The KTK slot also specifies the location of the preloaded KTK on the device

References

Possible Key Types:

Type	Name	Modifier
0x01	Master Session Key	0
0x02	DUKPT Initial Key	8
0x03	DUKPT BDK Key	8
0x04	MAC Key	3
0x05	PIN Encryption Key	1
0x06	Key Exchange Key	0
0x07	Host Verification KTK	0
0x08	DUKPT 3DES BDK Key	8
0x09	Default KTK	0
0x0A	Balance decryption Key	0
0x0B	TDR DUKPT BDK	0

Table 1: Possible Key Types for Futurex Protocol

NOTE: All BDK key types will not be injected. An initial key will be injected which is derived from the BDK and the KSN.